Experimental Analysis of Sorting Algorithms

Andrei Takacs

2025–2026

Abstract

Sorting is a basic but essential concept in computer science. This report looks at five classic sorting algorithms: Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort. The goal is to see how they actually perform in the real world compared to their theoretical speeds. I tested them using different input sizes (from 10 up to a million), different data types (integers, floats, and strings), and various starting orders (like random, sorted, or reversed). The results clearly show why $\mathcal{O}(N \log N)$ algorithms like Quick Sort are so much better for large datasets, while $\mathcal{O}(N^2)$ algorithms like Bubble Sort just give up and time out. I also found that sorting strings takes way longer than numbers, and that the initial shuffle of the data can really mess with Quick Sort's speed.

Contents

1 Introduction **2**

2 Scope of the Experiment **2**

3 Algorithms Under Study **2**
3.1 Bubble Sort 2
3.2 Insertion Sort 2
3.3 Merge Sort 3
3.4 Quick Sort 3
3.5 Heap Sort 3

4 Experimental Setup **3**

5 Results **3**

6 Analysis and Discussion **7**
6.1 The Big Difference in Scaling 7
6.2 Why Quick Sort Wins 7
6.3 Data Types Matter 7
6.4 How Starting Order Affects Things 7

7 Expectations vs. Reality **8**

8 Key Findings **8**

9 Conclusion **8**

10 Limitations / Future Work **8**

1 Introduction

Sorting data is something software does all the time, from database searches to data analysis. While we learn about Big-$\mathcal{O}$ notation in class to predict how fast an algorithm is, theory doesn't always match reality. Hardware details, memory limits, and the data itself can change everything. This report tests these algorithms in a C++ environment to see when a complex algorithm is actually worth the effort. By comparing standard slow algorithms with faster divide-and-conquer ones, we can see exactly how they handle different situations.

2 Scope of the Experiment

- **Input Sizes (N):** The tests scaled up by powers of ten: 10, 100, 1,000, 10,000, 100,000, and 1,000,000 elements.
- **Data Types:** The arrays had three different data types:
  - *Integers (`int`)*: Standard whole numbers, which are usually the fastest to compare.
  - *Floats (`float`)*: Decimal numbers.
  - *Strings (`string`)*: Words/characters, which take longer to compare because the computer has to check them letter by letter.
- **Data Distributions:** To see how the algorithms handle tricky situations, the data started in six different states: Random, Sorted, Reverse, Almost Sorted, Half Sorted, and Flat (lots of duplicates).
- **Algorithms Evaluated:** Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort.

3 Algorithms Under Study

Table 1 summarizes the core characteristics and history of the algorithms I tested.

Algorithm	**Best**	**Worst**	**Space**	**Stable**	**Year**
Bubble Sort	$\mathcal{O}(N)$	$\mathcal{O}(N^2)$	$\mathcal{O}(1)$	Yes	1956
Insertion Sort	$\mathcal{O}(N)$	$\mathcal{O}(N^2)$	$\mathcal{O}(1)$	Yes	1946
Merge Sort	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	Yes	1945
Quick Sort	$\mathcal{O}(N \log N)$	$\mathcal{O}(N^2)$	$\mathcal{O}(\log N)$	No	1959
Heap Sort	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(1)$	No	1964

Table 1: Algorithm Characteristics

3.1 Bubble Sort

A really simple algorithm that just steps through the list over and over, swapping adjacent items if they are in the wrong order. It's mostly just for learning and is way too slow for real work.

3.2 Insertion Sort

Builds the sorted list one item at a time by picking the next element and inserting it where it belongs. It's similar to Bubble Sort but usually does fewer swaps, making it pretty good for small or "almost sorted" lists.

3.3 Merge Sort

A reliable divide-and-conquer approach. It splits the array in half until it can't anymore, then merges the pieces back together in order. It's always fast, but it needs extra memory to hold the pieces while merging.

3.4 Quick Sort

Another divide-and-conquer method. It picks a "pivot" and moves everything smaller to the left and larger to the right, then repeats. It's usually the fastest in practice because it works well with computer memory, even if it can technically be slow in the worst-case scenario.

3.5 Heap Sort

Turns the array into a binary heap, then repeatedly pulls the largest item to the end. It doesn't need extra memory like Merge Sort, but it tends to be a bit slower than Quick Sort because of how it jumps around in memory.

4 Experimental Setup

I kept the testing environment strict to get accurate numbers:

- **Programming Environment:** I wrote and ran the code in C++, which lets it run right on the hardware without any extra background overhead like you'd get in Python or Java.
- **Timing Methodology:** I used standard C++ clocks (like `std::chrono`) to track exactly how long the sorting took.
- **Timeout Threshold:** To stop the really slow algorithms from freezing my computer forever, I set a timeout limit of 15s. If it took too long, I just recorded it as `TIMEOUT`.
- **Hardware Setup:** I ran this on my desktop PC, which has an AMD Ryzen 5 2600 processor and 16 GB of RAM. It's a standard everyday computer.

5 Results

Here are the visual results of the benchmarks. I used a log-log scale for the graphs because the numbers get huge very fast as the input size grows from 10 to 1 million.

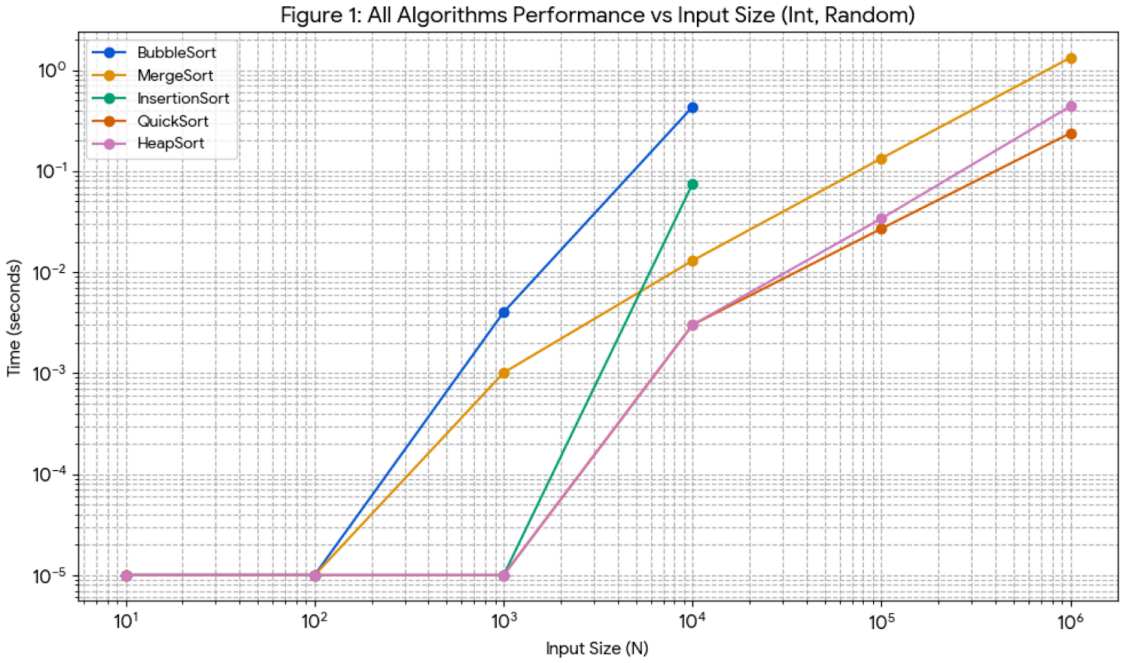

Figure 1: All Algorithms vs Input Size (Int, Random)

This first graph puts all five algorithms side-by-side. You can easily see the moment when the slow algorithms (Bubble and Insertion) just hit a wall around 10,000 to 100,000 elements, while the fast ones keep cruising.

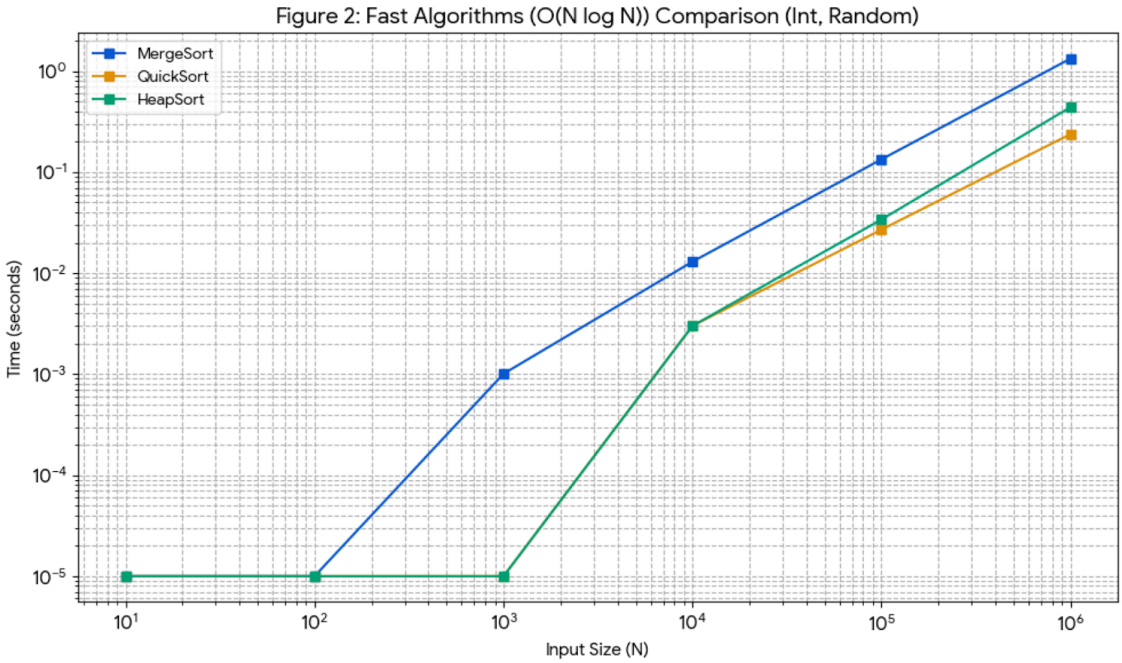

Figure 2: Fast Algorithms ($\mathcal{O}(N \log N)$) Comparison (Int, Random)

If we zoom in on just the fast algorithms, Quick Sort is the clear winner. At 1,000,000 items, it finished in about 0.238 seconds, beating Heap Sort (0.440s) and totally crushing Merge Sort (1.331s).

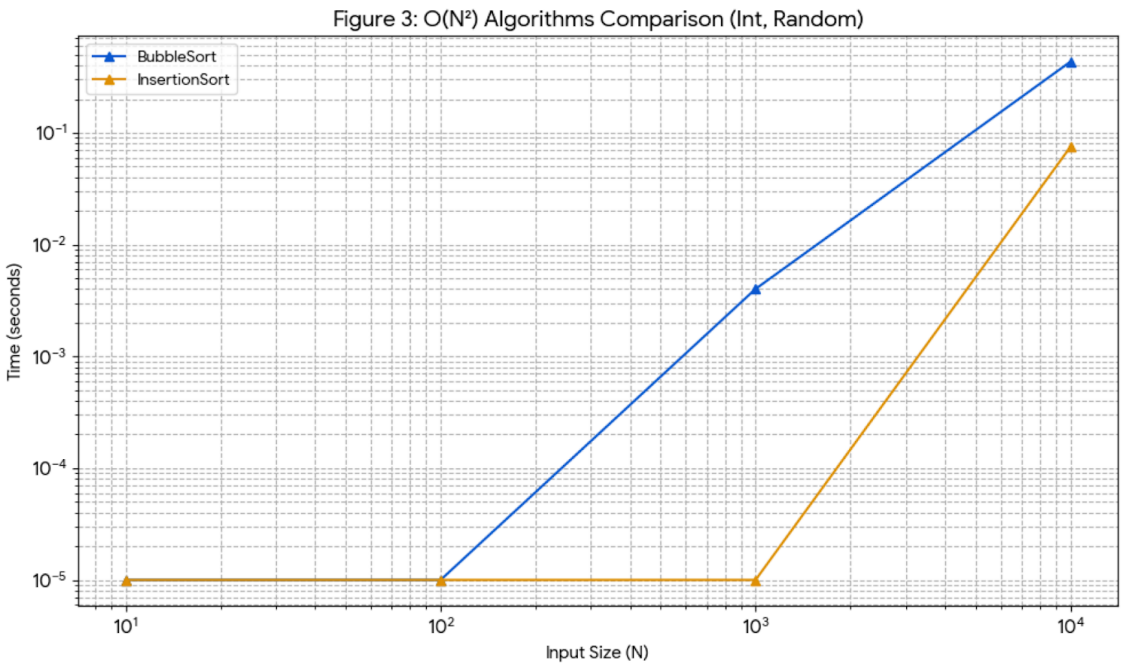

Figure 3: $\mathcal{O}(N^2)$ Algorithms Comparison (Int, Random)

Looking at just Bubble Sort and Insertion Sort, both have a really steep curve. They both completely timed out when trying to sort 100,000 items, proving they can't handle big data.

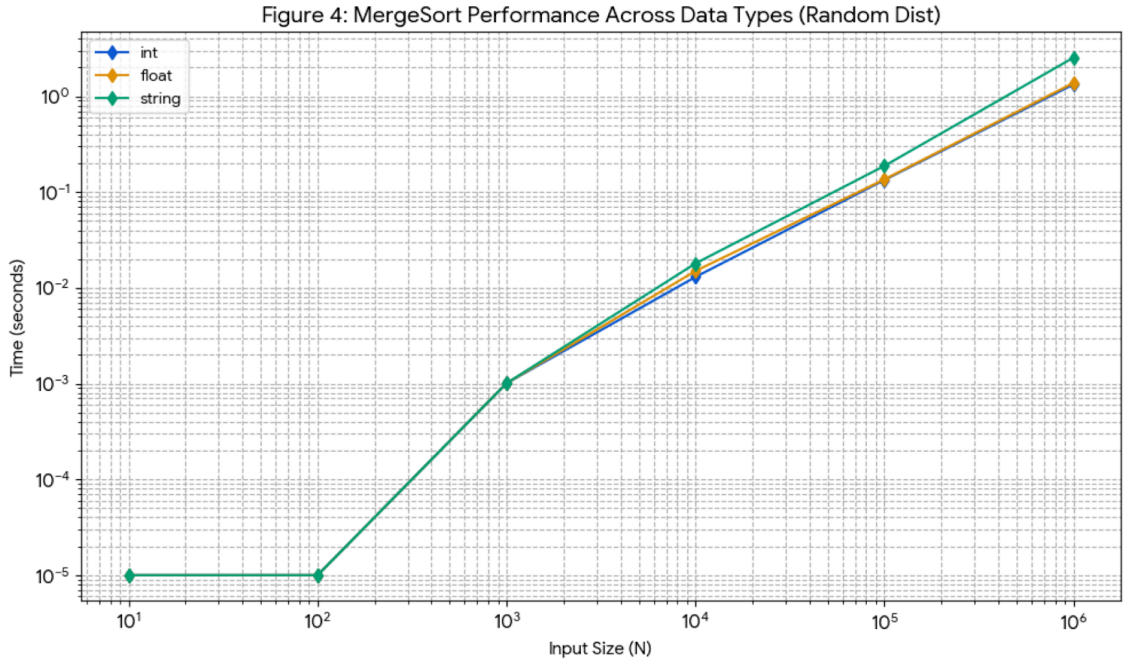

Figure 4: Merge Sort Performance Across Data Types (Random Dist)

This graph shows what happens when we use different data types but stick to one algorithm (Merge Sort). Integers and floats take almost the same amount of time, but strings take way longer because comparing text is a lot harder for the CPU.

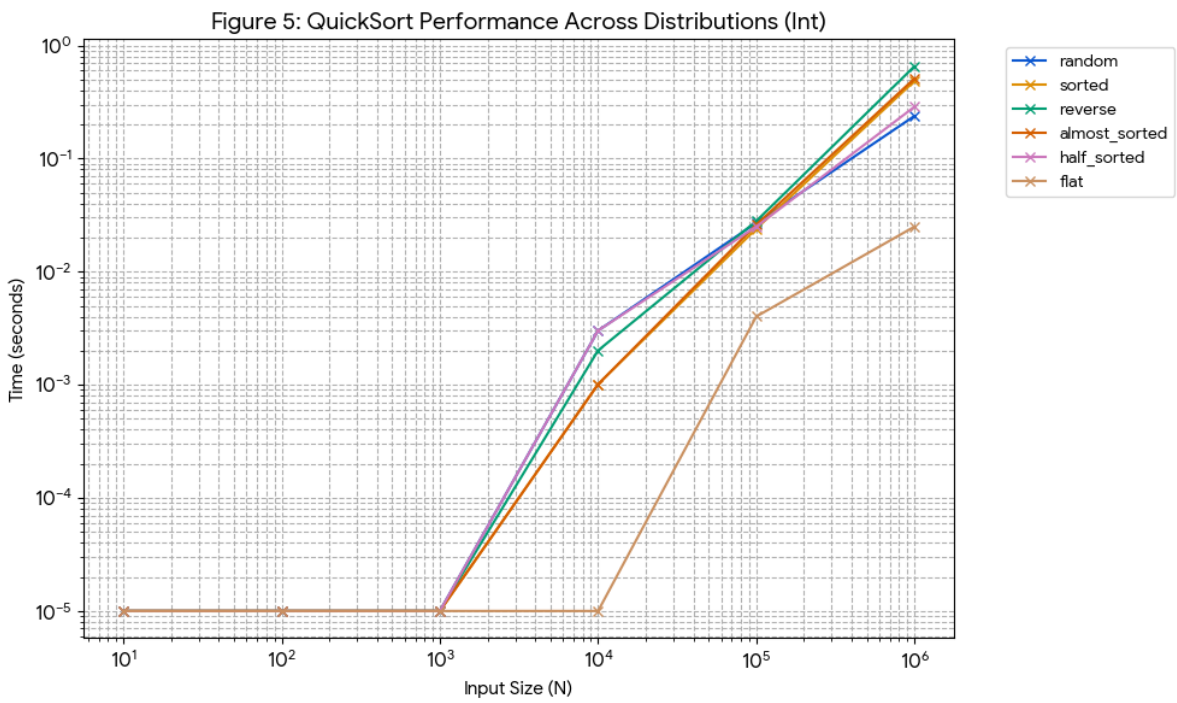

Figure 5: Quick Sort Performance Across Distributions (Int)

Finally, this chart shows how Quick Sort reacts to different initial setups. If an array is completely "flat" (lots of the same numbers), it finishes almost instantly. But if the array is in reverse order, it takes over twice as long as a random array.

6 Analysis and Discussion

6.1 The Big Difference in Scaling

The most obvious takeaway from the graphs is the massive gap between $\mathcal{O}(N^2)$ and $\mathcal{O}(N \log N)$ algorithms. For 10,000 items, Bubble Sort takes about 0.43 seconds. But because of how math works, trying to sort a million items wouldn't just take 100 times longer—it would take 10,000 times longer! That's why Insertion Sort and Bubble Sort timed out at 100,000 elements. They just aren't meant for big jobs.

6.2 Why Quick Sort Wins

Even though Quick Sort, Heap Sort, and Merge Sort all share an $\mathcal{O}(N \log N)$ average complexity, Quick Sort beat the others easily. It took just 0.238 seconds for a million items. This happens because Quick Sort is really good at working with the CPU's cache memory. Heap Sort jumps all over the place memory-wise, and Merge Sort wastes time creating and copying extra arrays.

6.3 Data Types Matter

Sorting isn't just about the number of items; it's about what those items are. Sorting a million strings took almost twice as long as sorting a million integers. Integers are simple for a computer to compare in one quick step, but strings require the computer to look at individual characters one by one.

6.4 How Starting Order Affects Things

Quick Sort showed some interesting quirks depending on how the data was shuffled to start with. When sorting an array where the numbers were mostly the same (flat), it flew through it in 0.025 seconds. But when given a list that was sorted completely backwards, it struggled, taking 0.655 seconds. This shows that picking a bad "pivot" can really slow down Quick Sort, even if it doesn't break it completely.

7 Expectations vs. Reality

What I Expected	What Actually Happened
Merge Sort, Quick Sort, and Heap Sort should all perform about the same since they are $\mathcal{O}(N \log N)$.	Quick Sort totally crushed the others. It was over 5 times faster than Merge Sort just because it handles memory better.
The time it takes to sort should mostly just depend on how many items there are.	The data type was huge. Sorting strings took almost twice as long as integers for the exact same amount of items.
Quick Sort is always incredibly fast across the board.	Quick Sort slowed down a lot (took 2.7x longer) when the data was given to it in reverse order.
The slow algorithms might still be okay for medium-sized lists.	They hit a hard wall. At 100,000 items, they just froze up and hit the timeout limit.

Table 2: Expectations vs. Reality

8 Key Findings

- **Slow Algorithms Don't Scale:** Bubble Sort and Insertion Sort become completely useless on a standard PC somewhere between 10,000 and 100,000 items.
- **Quick Sort is the Best:** Quick Sort is definitely the go-to algorithm. It finished 5.5x faster than Merge Sort for a million items.
- **Strings are Heavy:** Comparing words takes way more processing time than comparing simple numbers.
- **Shuffles Matter:** Quick Sort loves arrays with lots of duplicate numbers, but really doesn't like reverse-sorted arrays.

9 Conclusion

This project was a great way to see how algorithmic theory actually plays out in a real C++ program. The benchmarks proved that if you're sorting anything large on a standard computer like my Ryzen 5, Quick Sort is easily the best choice because it uses memory efficiently. On the flip side, trying to use Bubble Sort or Insertion Sort for anything bigger than a tiny list is just going to crash or freeze the program. In the end, picking the right algorithm isn't just about the number of items—you also have to think about what type of data it is and how messy it is to start with.

10 Limitations / Future Work

This experiment was pretty thorough, but it's not perfect. Since I wrote this in C++, the exact speeds might change depending on which compiler I use or if I turned on optimization flags. In the future, it would be cool to test how different compilers change the speed or the difference between different cpu arhitectures. Also, looking into exactly how many times the CPU missed its cache or writing a multi-threaded version of Merge Sort would be really interesting to see.